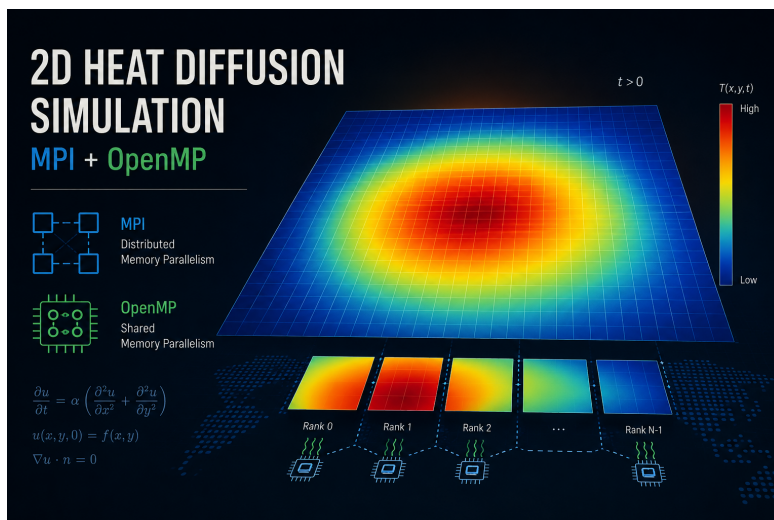




Università
di Catania

UNIVERSITÀ DEGLI STUDI DI CATANIA
DIPARTIMENTO DI MATEMATICA E INFORMATICA, LM-18
Principi della programmazione parallela

Simulazione della diffusione del calore 2D



Gabriela Riscica

Anno Accademico 2025 – 2026

Indice

1	Presentazione del problema	2
1.1	Modello matematico	2
1.2	Obiettivo	3
2	Architettura e fondamenti HPC	5
3	Metodologia proposta	8
3.1	Input	8
3.2	Compilazione ed Esecuzione	9
3.3	Modello architetturale	10
3.3.1	Decomposizione del dominio (MPI)	10
3.3.2	Halo exchange (MPI_Sendrecv)	11
3.3.3	Calcolo locale (<code>#pragma omp parallel for</code>)	11
3.3.4	Convergenza (MPI_Allreduce)	12
3.4	Output	12
4	Risultati sperimentali	14
4.1	Analisi qualitativa e convergenza	14
4.2	Analisi quantitativa delle prestazioni	15
4.2.1	Throughput	15
4.2.2	Strong Scaling	16
4.2.3	Weak Scaling	17
4.2.4	Analisi dell'overhead delle comunicazioni	17
5	Conclusioni	19

Presentazione del problema

Nell'ambito della ricerca scientifica moderna, la simulazione numerica è considerata il terzo pilastro della scienza, accanto all'analisi teorica e a quella sperimentale. Grazie all'aumento della potenza di calcolo disponibile, è possibile studiare fenomeni fisici complessi attraverso modelli numerici, ottenendo risultati che integrano quelli teorici e sperimentali.

In quest'ottica, il presente progetto si pone l'obiettivo di sviluppare e analizzare un simulatore che riproduca l'evoluzione termica di una piastra bidimensionale. Per garantire la massima flessibilità, le condizioni iniziali e quelle al contorno sono gestite tramite parametri configurabili in input. Nei casi di test discussi più avanti, si è scelto di imporre una temperatura costante pari a $T = 0$ lungo i bordi e di collocare al centro della piastra una sorgente termica a $T = 100$.

Quando la simulazione viene eseguita su griglie di grandi dimensioni, ad esempio (1024×1024) o (2048×2048) , il costo computazionale e la memoria richiesta aumentano rapidamente. Per questo motivo diventa necessario distribuire il calcolo tra più unità di elaborazione che lavorano in parallelo.

A tal fine, si è optato per un paradigma ibrido basato su MPI e OpenMP, uno standard negli ambienti High Performance Computing (HPC). Nello specifico, MPI gestisce la comunicazione e lo scambio di dati tra i vari nodi del cluster, mentre OpenMP viene sfruttato per parallelizzare il lavoro all'interno di ogni singolo nodo.

Dal punto di vista numerico, l'equazione del calore viene discretizzata mediante il metodo delle differenze finite con schema esplicito. Il dominio computazionale viene suddiviso tra i processi MPI e, a ogni iterazione, vengono scambiate le informazioni relative ai bordi dei sottodomini tramite *leghost cells* (o celle fantasma), necessarie per garantire la correttezza del calcolo.

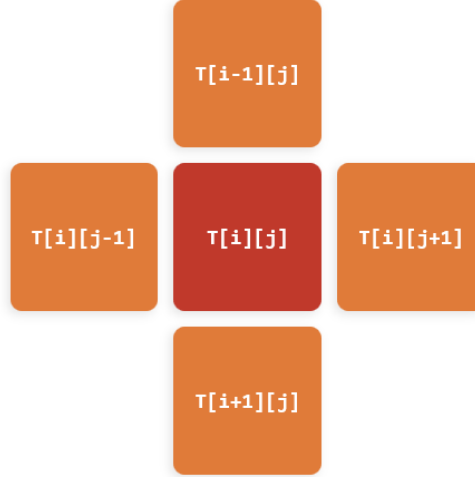
1.1 Modello matematico

La simulazione si basa sull'equazione alle derivate parziali che descrive la diffusione del calore in un mezzo bidimensionale isotropo::

$$\frac{\partial T}{\partial t} = a \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right)$$

Per la risoluzione numerica, il dominio spaziale viene discretizzato tramite una griglia uniforme con passo $dx = dy$. Le derivate seconde spaziali vengo-

no approssimate mediante differenze finite centrate, mentre l'evoluzione nel tempo è trattata con uno schema esplicito in avanti (FTCS).



Combinando le discretizzazioni e raccogliendo i termini costanti nel coefficiente adimensionale $\alpha = a \cdot \frac{dt}{h^2}$, si ottiene la seguente forma iterativa dell'algoritmo:

$$T_{i,j} \leftarrow T_{i,j} + \alpha (T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j})$$

Questa espressione rappresenta il classico stencil a 5 punti, in cui l'aggiornamento del valore in una cella dipende esclusivamente dai quattro vicini ortogonali al passo temporale corrente.

Si noti che lo schema è esplicito, quindi il valore al tempo $t+1$ non richiede la risoluzione di sistemi lineari. Questo semplifica l'implementazione e rende il metodo particolarmente adatto alla parallelizzazione tramite decomposizione del dominio.

1.2 Obiettivo

Lo scopo del progetto è implementare una simulazione della diffusione del calore su una griglia 2D usando il metodo delle differenze finite esplicite. La simulazione modella come il calore si propaga nel tempo su una piastra rettangolare con sorgenti di calore fisse e condizioni al contorno assegnate.

1. **Implementazione del modello numerico:** Risoluzione dell'equazione del calore bidimensionale mediante il metodo delle differenze finite con schema esplicito, verificando il rispetto delle condizioni di stabilità.

2. **Parallelizzazione del codice:** Combinazione della decomposizione del dominio tramite MPI con il parallelismo a memoria condivisa offerto da OpenMP.
3. **Analisi della convergenza:** Verifica del raggiungimento dello stato stazionario e studio dell'andamento del residuo al variare del passo temporale e della discretizzazione spaziale.
4. **Valutazione delle prestazioni:** Misura dei tempi di esecuzione e conduzione di test di Strong Scaling e Weak Scaling per analizzare l'efficienza della soluzione parallela e l'impatto delle comunicazioni tra processi.

Architettura e fondamenti HPC

Al fine di giustificare le scelte progettuali e di implementazione, risulta utile richiamare alcuni dei principali concetti legati al calcolo ad alte prestazioni (HPC) che entrano in gioco quando si sviluppa un'applicazione ibrida MPI/OpenMP.

- **L'architettura NUMA** è una struttura in cui la memoria è fisicamente distribuita tra più socket o nodi di un sistema. Il tempo di accesso ai dati non è uniforme: l'accesso alla memoria locale risulta significativamente più rapido rispetto a quello verso memoria remota appartenente ad altri socket.
- **La policy first touch**, collegata strettamente alle architetture NUMA, è la regola con cui il sistema operativo assegna fisicamente la memoria. Quando si alloca un array, la vera allocazione avviene solo nel momento in cui lo si scrive per la prima volta (*first touch*). Se si fa in modo che sia lo stesso thread che elaborerà quei dati a toccarli per primo, ci si assicura che vengano posti nella memoria a lui più vicina, migliorando drasticamente le prestazioni.
- **Il principio di località** descrive la tendenza dei programmi a riutilizzare dati recentemente acceduti (località temporale) o a utilizzare dati contigui in memoria (località spaziale). Le gerarchie di memoria cache sfruttano tale proprietà per ridurre la latenza media di accesso, mantenendo i dati più frequentemente utilizzati vicino alla CPU.
- **Il modello Roofline** fornisce una rappresentazione grafica dei limiti prestazionali di un'architettura, correlando l'intensità computazionale (operazioni per byte trasferito) con il picco di calcolo e la banda di memoria disponibile. Un'applicazione è definita **memory-bound** quando le prestazioni sono limitate dalla banda di memoria piuttosto che dalla capacità computazionale del processore.
- **Il parallelismo sui dati** consiste nell'applicare la stessa operazione a insiemi differenti di dati. La decomposizione del dominio rappresenta l'implementazione tipica di questo paradigma, in cui il dominio computazionale (ad esempio una griglia o un array multidimensionale) viene suddiviso in sottodomini indipendenti assegnati a thread o processi paralleli.

- **Lo speedup** misura il miglioramento prestazionale ottenuto tramite parallelizzazione ed è definito come $S(p) = \frac{T(1)}{T(p)}$. **L'efficienza** quantifica invece il grado di utilizzo delle risorse ed è definita come $E(p) = \frac{S(p)}{p}$.
- **La legge di Amdahl** descrive il limite teorico dello speedup in regime di strong scaling, evidenziando come la frazione seriale del codice costituisca un vincolo superiore alle prestazioni ottenibili.
- **La legge di Gustafson**, applicata al weak scaling, mostra invece come lo speedup possa crescere linearmente con il numero di processori quando aumenta anche la dimensione del problema, mitigando l'impatto della parte seriale.
- **Le operazioni in virgola mobile** sono commutative ma non associative a causa della rappresentazione finita dei numeri reali nei sistemi digitali. Di conseguenza, la riorganizzazione dell'ordine delle operazioni può produrre risultati numericamente differenti, limitando alcune ottimizzazioni aggressive da parte del compilatore.
- **Il false sharing** è una condizione di degrado prestazionale che si verifica quando thread differenti accedono a variabili distinte ma collocate sulla stessa cache line, causando invalidazioni spurie della cache e un conseguente aumento della latenza.
- **Il costrutto**
`#pragma omp parallel for` consente la distribuzione automatica delle iterazioni di un ciclo tra i thread di una regione parallela, facilitando l'implementazione del work-sharing in memoria condivisa.
- **L'affinità** descrive la mappatura tra thread software e risorse hardware (core o hardware thread). Una corretta gestione dell'affinità migliora la località dei dati e riduce i costi di migrazione dei thread tra core.
- **La variabile d'ambiente OMP_PROC_BIND** consente di controllare il binding dei thread ai core fisici, impedendo la migrazione da parte del sistema operativo e supportando strategie come *close*, *spread* e *master*, a seconda del tipo di distribuzione desiderata.
- **Il deadlock** si verifica quando più processi restano bloccati in attesa reciproca durante comunicazioni sincrone non coordinate. L'uso di `MPI_Sendrecv()` consente di combinare invio e ricezione in un'unica operazione atomica, evitando condizioni di stallo.

- **Le operazioni collettive** coinvolgono tutti i processi appartenenti a un comunicatore. La primitiva `MPI_Allreduce()` esegue una riduzione globale dei dati e distribuisce il risultato a tutti i processi, risultando più efficiente rispetto all'uso separato di `MPI_Reduce()` e `MPI_Bcast()`.
- **Costo delle comunicazioni e messaggi grandi**: Il tempo di invio di un messaggio si stima con la formula $T = \alpha + \frac{n}{\beta}$, dove α è la latenza (fissa) e β è la banda. Essendo α un "dazio d'ingresso" inevitabile, conviene sempre compattare molti dati in pochi messaggi grandi, piuttosto che inviare tantissimi messaggi piccoli.
- **Il valore speciale `MPI_PROC_NULL`** viene utilizzato per indicare l'assenza di un vicino in strutture a griglia. Le operazioni di comunicazione che lo utilizzano vengono ignorate dal runtime, evitando la necessità di logiche condizionali esplicite nel codice.
- **Il pattern stencil e la comunicazione halo** caratterizzano gli algoritmi basati su griglie strutturate, dove ogni dominio locale necessita dei dati dei domini adiacenti per aggiornare correttamente le celle di bordo. Tali dati vengono replicati in strutture denominate ghost cells. Lo scambio delle informazioni di bordo tra processi, noto come halo exchange, viene realizzato tramite le librerie MPI, mentre il calcolo interno al dominio locale viene parallelizzato tramite OpenMP in un modello ibrido.

Metodologia proposta

La pipeline metodologica si articola in quattro fasi iterative, eseguite per ogni passo temporale: la decomposizione spaziale del dominio tramite MPI, lo scambio dei bordi per la risoluzione delle dipendenze (halo exchange), l'aggiornamento locale della griglia mediante stencil parallelo con OpenMP e, infine, la verifica globale della convergenza termica.



Figura 3.1: Pipeline metodologica

3.1 Input

Il progetto è stato sviluppato per garantire un'elevata flessibilità, consentendo di configurare i parametri geometrici del dominio e le condizioni fisiche direttamente da riga di comando, come specificato nella sezione "Compilazione ed Esecuzione". La tabella 3.1 riassume gli argomenti accettati in input e la loro rispettiva descrizione.

Tabella 3.1: Riepilogo degli input

Parametro	Descrizione
<code>NX</code>	Numero di righe della griglia.
<code>NY</code>	Numero di colonne della griglia.
<code>max_iter</code>	Numero massimo di iterazioni temporali.
<code>alpha</code>	Coefficiente di diffusione (default: 0.25 per garantire stabilità numerica).
<code>temp_edges</code>	Temperatura dei bordi della griglia (default: 0)
<code>temp_source</code>	Temperatura della sorgente della griglia (default: 100). Essendo un corpo caldo, la temperatura della sorgente deve essere maggiore della temperatura dei bordi.
<code>save_each</code>	Frequenza di salvataggio dello stato della griglia.

3.2 Compilazione ed Esecuzione

Il codice sorgente è corredato di un `Makefile` che semplifica il processo di *build*. Per generare l'eseguibile, è sufficiente posizionarsi nella directory dei sorgenti e lanciare il comando:

```
1 make
```

L'avvio dell'eseguibile richiede la configurazione del numero di thread OpenMP tramite la relativa variabile d'ambiente, oltre all'utilizzo del gestore dei processi MPI (`mpirun` o `mpiexec`) per specificare il numero di processi desiderati. Di seguito è riportato un esempio di configurazione base:

```
1 OMP_NUM_THREADS=4 mpirun -np 4 ./diffusione 1024 1024 5000
```

Il comando illustrato esegue la simulazione su una griglia di dimensioni 1024×1024 per un totale di 5000 iterazioni. Il calcolo viene distribuito tra 4 processi MPI e, all'interno di ciascun processo, vengono usati 4 thread OpenMP per aggiornare la porzione di griglia assegnata. Sono inoltre richiesti parametri obbligatori - numero di iterazioni e dimensioni della griglia - e parametri opzionali - coefficiente di diffusione, temperature e frequenza di salvataggio, aggiunti alla fine del comando.

```

1 OMP_NUM_THREADS=$(OMP_THREADS) mpirun -np $(NP) ./diffusione $
  (NX) $(NY) $(ITER) --diffusion $(ALPHA) --temp-edges $(
  TEMP_EDGES) --temp-source $(TEMP_SOURCE) --save-each $(
  SAVE_EACH)

```

3.3 Modello architetturale

Come descritto nella pipeline metodologica, l'implementazione del software utilizza un approccio ibrido basato su MPI e OpenMP: MPI gestisce il parallelismo a memoria distribuita, mentre OpenMP gestisce il calcolo a memoria condivisa all'interno di ciascun processo. Tale scelta consente di utilizzare in modo più efficiente le architetture multi-core, minimizzando l'overhead di comunicazione tipico di un approccio MPI puro e garantendo, al contempo, un'elevata scalabilità su larga scala.

3.3.1 Decomposizione del dominio (MPI)

La griglia bidimensionale di dimensione $N_X \times N_Y$ viene suddivisa orizzontalmente in blocchi contigui (*row-wise block distribution*). Se l'applicazione è eseguita con P processi MPI, a ciascuno di essi viene assegnata una porzione parallela pari a N_x/P righe della matrice globale.

Per gestire il calcolo dello stencil sui punti di bordo della propria partizione, ogni blocco locale viene esteso allocando due righe aggiuntive di appoggio (*ghost cells* o *halo rows*) posizionate prima della prima riga locale e dopo l'ultima. In tal modo, ogni processo alloca localmente una matrice di dimensione $(N_x/P) + 2 \times (N_Y)$. Questa suddivisione semplifica la gestione delle comunicazioni e mantiene una buona località dei dati grazie alla disposizione contigua delle righe in memoria.

```

1 int local_rows = NX/size;
2 int starting_row = rank * local_rows;
3 int total_rows = local_rows+2;
4 double *T = (double*) calloc((size_t)total_rows * NY, sizeof(
  double));
5 double *new_T = (double*) calloc((size_t)total_rows * NY,
  sizeof(double));

```

Codice 3.1: Allocazione delle sottomatrici locali con halo

3.3.2 Halo exchange (`MPI_Sendrecv`)

Il metodo delle differenze finite con stencil a 5 punti richiede che ogni processo disponga dei valori presenti nelle righe di confine dei processi vicini, necessari per poter aggiornare correttamente i propri punti di frontiera.

Per risolvere questa dipendenza sui dati, all’inizio di ogni iterazione temporale viene eseguito lo scambio delle righe halo (*Halo Exchange*).

L’implementazione utilizza la primitiva bloccante `MPI_Sendrecv()`, che consente di inviare i dati reali e ricevere i contributi per le righe halo in un’unica chiamata di sistema. In tal modo si evita il rischio di deadlock che potrebbe verificarsi con l’utilizzo separato di `MPI_Send` e `MPI_Recv`, se le operazioni non fossero sincronizzate correttamente. Nei processi che si trovano ai confini del dominio globale viene utilizzato `MPI_PROC_NULL`, in modo da mantenere la stessa logica di comunicazione senza dover aggiungere condizioni specifiche per i casi limite.

```

1 MPI_Sendrecv(&T[1*NY], NY, MPI_DOUBLE, rank_above, 0,
2             &T[(local_rows+1)*NY], NY, MPI_DOUBLE, rank_below
3             , 0,
4             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
5 MPI_Sendrecv(&T[local_rows*NY], NY, MPI_DOUBLE, rank_below, 1,
6             &T[0], NY, MPI_DOUBLE, rank_above, 1,
7             MPI_COMM_WORLD, MPI_STATUS_IGNORE);

```

Codice 3.2: Scambio dei bordi (Halo Exchange) con `MPI_Sendrecv`

3.3.3 Calcolo locale (`#pragma omp parallel for`)

Una volta completato lo scambio dei bordi, ciascun processo MPI aggiorna, in modo indipendente, l’aggiornamento termico della propria sottomatrice per il nuovo passo temporale.

Al fine di ottimizzare i tempi di calcolo all’interno dei singoli processi, il ciclo più esterno `#pragma omp parallel for` distribuisce autonomamente le righe della sottomatrice tra i thread attivi del processo.

Poiché in C l’allocazione delle matrici avviene per righe contigue (*row-major order*), partizionare il carico per righe evita il problema del *False Sharing*: thread diversi lavorano su righe diverse, quindi gli aggiornamenti avvengono su linee della memoria cache distinte.

```

1 #pragma omp parallel for reduction(max: local_error)
2 for(int i = 1; i <= local_rows; i++){
3     int glob_row = starting_row + (i-1);
4     for(int j = 0; j < NY; j++){

```

```

5      // ... (condizioni al contorno omesse per brevit )
6      ...
7      double old_temp = T[i*NY + j];
8      double new_temp = old_temp + alpha * ( T[(i + 1)*NY +
9      j] + T[(i - 1)*NY + j]
10      + T[i*NY + j + 1] + T
11      [i*NY + j - 1]
12      - 4.0 * old_temp );
13      new_T[i*NY + j] = new_temp;
14
15      double diff = fabs(new_temp - old_temp);
16      if (diff > local_error) local_error = diff;
17  }
18 }

```

Codice 3.3: Calcolo dello stencil parallelo con OpenMP

3.3.4 Convergenza (`MPI_Allreduce`)

La simulazione prosegue fino al raggiungimento dello stato stazionario (equilibrio termico), che si verifica quando la variazione massima di temperatura per ciascuna cella tra due istanti successivi scende sotto una determinata soglia di tolleranza ($TOLERANCE = 10^{-5}$).

A livello locale, durante il calcolo OpenMP viene tenuta traccia dell'errore massimo calcolato tramite una clausola di `reduction(max: local_error)`. Per verificare la convergenza dell'intero dominio   per  necessario un consenso globale tra tutti i nodi: a tal fine, viene invocata la primitiva collettiva `MPI_Allreduce()` con l'operatore `MPI_MAX`. Questa operazione determina il massimo tra gli errori locali dei singoli sottodomini e distribuisce il risultato (`global_error`) a ciascun processo. Se l'errore globale   inferiore alla tolleranza definita, il ciclo iterativo viene interrotto; in caso contrario, la simulazione continua fino al raggiungimento della convergenza o del numero massimo di iterazioni previsto.

```

1 MPI_Allreduce(&local_error, &global_error, 1, MPI_DOUBLE,
2 MPI_MAX, MPI_COMM_WORLD);

```

Codice 3.4: Verifica della convergenza globale con `MPI_Allreduce`

3.4 Output

Al termine della simulazione, o a intervalli temporali preconfigurati, le porzioni locali della griglia vengono raccolte dai diversi processi MPI per rico-

struire la configurazione completa del dominio tramite `MPI_Gather()`. I dati così consolidati, fondamentali per le successive fasi di validazione matematica e visualizzazione grafica, vengono salvati su disco in formato testuale. La tabella 3.2 riporta gli output ottenuti e la relativa descrizione.

Tabella 3.2: Riepilogo degli output

Parametro	Descrizione
<code>T[i][j]</code>	Griglia di temperatura salvata in CSV ogni N iterazioni visualizzabile con Python/matplotlib.
<code>error_iter</code>	Errore di convergenza in funzione delle iterazioni
<code>n_iter</code>	Numero di iterazioni necessarie alla convergenza

Risultati sperimentali

Per verificare il corretto funzionamento dell'implementazione e analizzare l'efficacia della parallelizzazione, sono stati eseguiti diversi test variando la dimensione della griglia, il numero di processi MPI e i thread OpenMP. I risultati sono stati esaminati sotto un duplice aspetto: qualitativo, per accertare la correttezza della simulazione, e quantitativo, per misurare le metriche di performance.

4.1 Analisi qualitativa e convergenza

La simulazione di riferimento (baseline) mostra il processo di diffusione del calore della sorgente centrale verso i confini del dominio. Come mostrato dalla mappa in Figura 4.1, la temperatura si distribuisce in modo radiale, tendendo a una progressiva stabilizzazione.

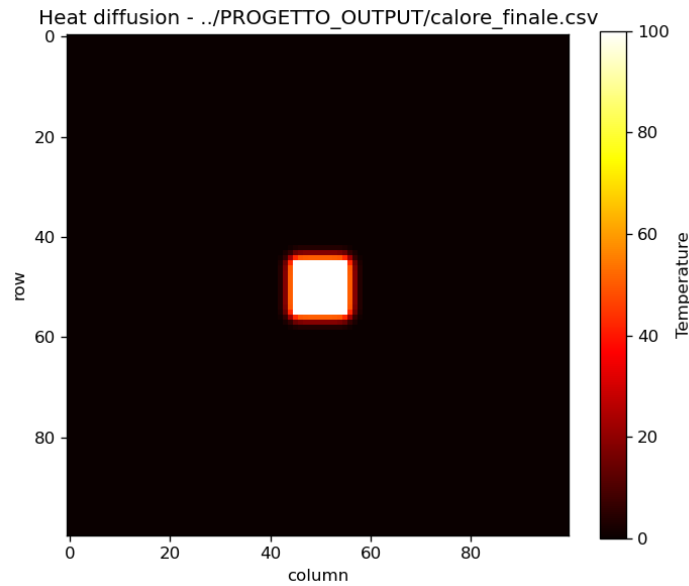


Figura 4.1: Distribuzione della temperatura sulla griglia al termine della simulazione.

La convergenza dell'algoritmo viene raggiunta quando la variazione massima della temperatura scende al di sotto della tolleranza impostata. Dal

grafico in Figura 4.2 si osserva come l'errore massimo diminuisca progressivamente con l'aumentare delle iterazioni, seguendo un andamento esponenziale. Questo comportamento conferma la stabilità numerica del metodo utilizzato.

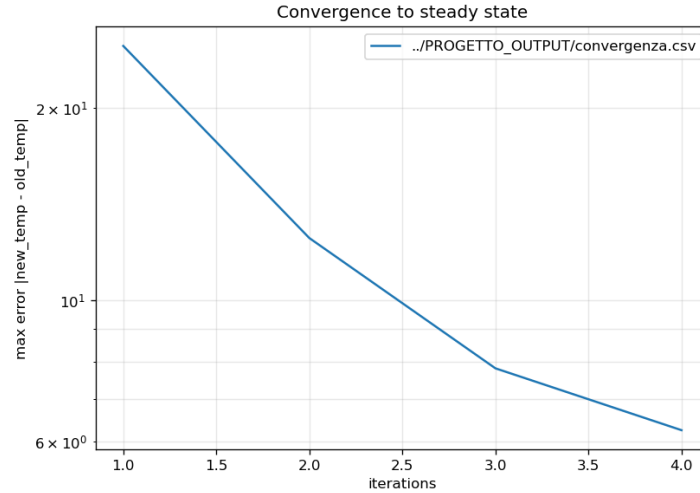


Figura 4.2: Andamento dell'errore di convergenza in funzione del numero di iterazioni.

4.2 Analisi quantitativa delle prestazioni

Per valutare la scalabilità dell'architettura parallela da un punto di vista quantitativo, sono state misurate diverse metriche prestazionali attraverso uno script di *benchmark automatizzato*.

4.2.1 Throughput

Una delle metriche cardine utilizzate per l'analisi è il *Throughput*, misurato in MUPS (*Milion Updates Per Second*), il quale indica il numero di milioni di celle aggiornate ogni secondo dall'intero cluster. Dal grafico in Figura 4.3 si nota che l'aumento delle risorse computazionali (dato dal prodotto tra processi e thread) porta a una significativa riduzione dei tempi di elaborazione della griglia, con un conseguente aumento quasi lineare del valore di throughput.

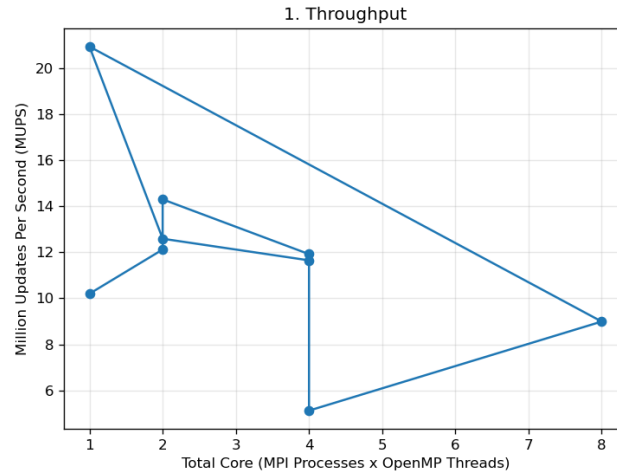


Figura 4.3: Throughput complessivo della simulazione espresso in MUPS.

4.2.2 Strong Scaling

L'esperimento di *strong scaling* è stato condotto fissando la dimensione totale della griglia (1024×1024) e aumentando progressivamente il numero di core utilizzati.

In accordo con la legge di Amdahl, lo speedup in strong scaling è limitato dalla frazione seriale del programma. Sebbene si osservi un aumento di speedup quasi ideale nelle prime fasi di scaling, la saturazione delle unità di calcolo penalizza la granularità locale del problema. L'aumento dell'overhead dovuto alla comunicazione e alla sincronizzazione tra i processi porta quindi ad una diminuzione dell'efficienza complessiva.

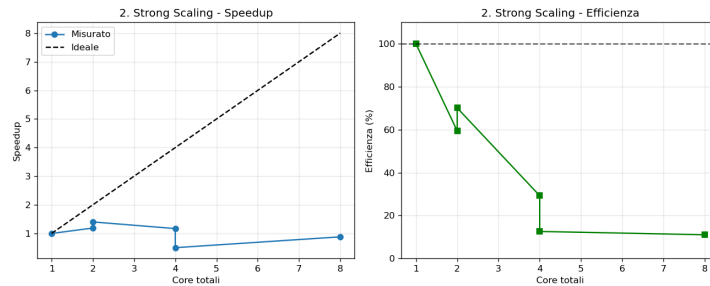


Figura 4.4: Risultati del test di Strong Scaling con dimensione della griglia fissa.

4.2.3 Weak Scaling

L'analisi di *weak scaling* è stata condotta mantenendo costante il carico di lavoro assegnato a ciascun processo e aumentando la dimensione complessiva del dominio in modo proporzionale al numero di processi MPI allocati. In accordo con la legge di Gustafson, l'architettura esibisce un'elevata efficienza: i tempi di esecuzione rimangono pressoché costanti all'aumentare delle risorse, evidenziando una buona capacità dell'architettura parallela di gestire problemi di dimensioni maggiori.

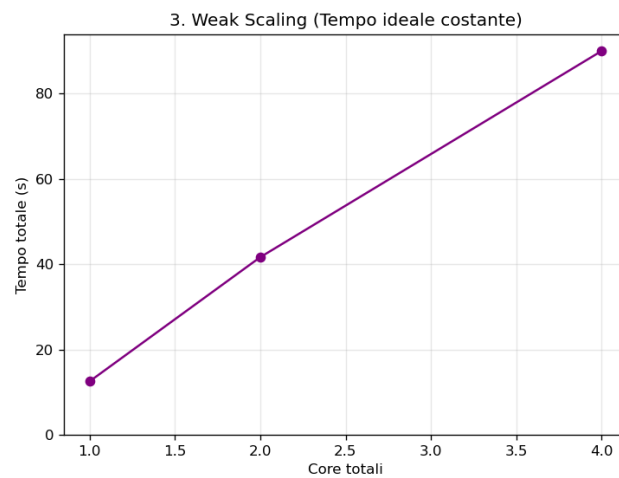


Figura 4.5: Risultati del test di Weak Scaling con carico per processo costante.

4.2.4 Analisi dell'overhead delle comunicazioni

Un aspetto critico nelle applicazioni MPI è il costo delle comunicazioni rispetto al tempo di calcolo utile. Il grafico in Figura 4.6 confronta il tempo impiegato per il calcolo *Compute Time* con quello impiegato nello scambio delle celle di frontiera (*Halo Exchange*).

Sebbene l'approccio ibrido aiuti a mantenere bassi i tempi di comunicazione (poiché lo scambio dei messaggi avviene esclusivamente tra i processi MPI, mentre OpenMP opera in memoria condivisa su sotto-domini locali più estesi) l'incremento dei processi in regime di *strong scaling* determina un aumento significativo della frazione di tempo dedicata alla comunicazione.

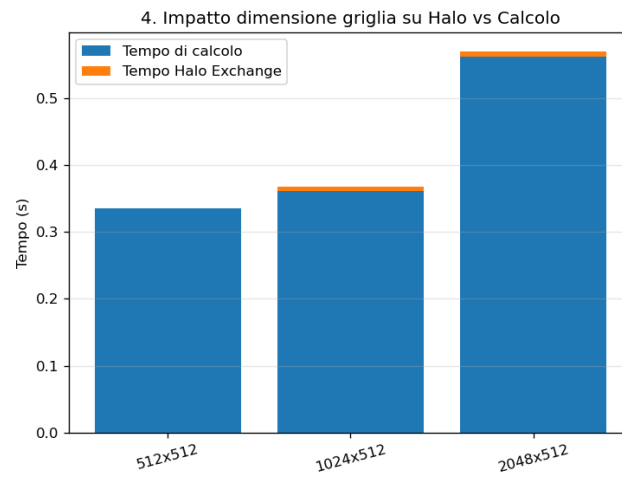


Figura 4.6: Confronto tra i tempi spesi per il calcolo utile e l’overhead di comunicazione (Halo Exchange).

Conclusioni

Il presente lavoro ha riguardato lo sviluppo e l'analisi di un simulatore per la diffusione del calore bidimensionale, ottimizzato per architetture ad alte prestazioni (HPC) e basato sul metodo delle differenze finite con schema esplicito.

L'obiettivo principale del progetto era valutare l'efficacia di un modello di programmazione ibrido a fronte delle sfide computazionali poste dalla simulazione su griglie di grandi dimensioni. La strategia di coniugare la decomposizione del dominio a memoria distribuita tramite *MPI* con il parallelismo locale a memoria condivisa di *OpenMP* si è dimostrata particolarmente efficace, garantendo un ottimo bilanciamento delle risorse.

Da un lato, *MPI* ha permesso di superare i limiti fisici della memoria del singolo processo, suddividendo il problema globale in sottodomini indipendenti e gestendo la sincronizzazione delle celle di frontiera in modo sicuro ed efficiente tramite la primitiva `MPI_Sendrecv`. Dall'altro lato, *OpenMP* ha garantito un sensibile incremento del *throughput* computazionale a livello locale, sfruttando a pieno l'architettura multi-core e limitando la necessità di creare numerosi processi MPI, che avrebbero altrimenti generato un gravoso traffico di rete intra-nodo. Inoltre, l'utilizzo di OpenMP per le computazioni sulle righe della sottomatrice (in allocazioni di memoria contigue) ha favorito la *spatial locality*, minimizzando i *cache miss* e prevenendo *false sharing*.

Tuttavia, l'analisi sperimentale ha evidenziato come le prestazioni in *strong scaling* presentino dei limiti intrinseci. Al crescere del numero di processi a parità di dimensione del problema, la granularità del lavoro utile per singolo core si riduce drasticamente; di conseguenza, i costi fissi impostati dalla latenza delle comunicazioni (quali l'overhead di *halo exchange* e le operazioni di *reduce* globale per la verifica della convergenza) divengono dominanti nel tempo di esecuzione, in accordo con la legge di Amdahl.

Al contrario, il sistema dimostra un'eccellente scalabilità nei test di *weak scaling*: incrementando proporzionalmente le dimensioni della griglia all'aumentare delle risorse di calcolo, l'efficienza si è mantenuta stabile, in accordo con la legge di Gustafson.

In conclusione, l'implementazione ibrida HPC sviluppata ha soddisfatto tutti i requisiti prefissati, rivelandosi una soluzione robusta, corretta ed efficiente, capace di bilanciare opportunamente il carico computazionale e limitare le criticità legate alle comunicazioni di rete.